

Low-Level System Monitoring Tool

Technical Architecture and Implementation Report for Arch Linux

Prepared by:

El Mahdi Chakor

Fourth-Year Computer Engineering Student

June 2026

Contents

1	Abstract	1
2	Introduction and Project Motivation	1
3	System Architecture and Virtual Kernel Interfaces	1
3.1	Mathematical Model for Real-Time CPU Profiling	1
3.2	Memory Mapping and Network Streaming Mechanisms	2
4	Modular Software Engineering Layout	2
5	Low-Level Graphics Deployment via Ncurses	2
6	Compilation Management and Strict Diagnostic Controls	2
6.1	Case Study: Silencing Unused Parameter Warnings under Strict Rules . .	2
6.2	Production Makefile Script	3
7	Conclusion and Next Phase Systems Engineering	3

1 Abstract

This report details the technical architecture, operational mechanisms, and implementation of a lightweight, low-level real-time system monitoring utility custom-engineered for Arch Linux. Operating entirely within user space without relying on heavy external performance frameworks, the application interfaces directly with the Linux Kernel via the `/proc` pseudo-filesystem. By pulling raw structures for CPU jiffies, memory states, and network device metrics, the program computes system health metrics under strict operational constraints. The visual frame is deployed via the `ncurses` graphics terminal layer, providing an industrial-grade workspace display.

2 Introduction and Project Motivation

Modern diagnostic tooling often introduces non-trivial system footprints, running comprehensive software runtimes just to map performance overheads. In performance-critical or embedded contexts, such layers distort actual system benchmarks.

The primary motivation of this project is to bypass intermediate abstraction layers and construct a native tracking interface. By operating at the boundary between user space and kernel facilities using the standard C compiler, this tool establishes a lightweight paradigm for tracking CPU, RAM, and Network states. This implementation serves as an intensive exploration of Linux memory paradigms, systems software architecture, and modular terminal graphics management.

3 System Architecture and Virtual Kernel Interfaces

The application rejects high-level monitoring APIs, interacting instead with the Linux Virtual Filesystem (VFS) by scraping kernel abstractions exposed via the `/proc` directory. These pseudo-files act as direct structural views into real-time kernel memory data structures.

3.1 Mathematical Model for Real-Time CPU Profiling

To capture accurate processing loads without relying on monolithic scheduling functions, the software extracts raw state metrics from `/proc/stat`. Individual CPU state fields (user, nice, system, idle, iowait, irq, and softirq) are read across a precise sampling interval (Δt). The percentage load is calculated using the following mathematical model:

$$\text{TotalJiffies} = \text{user} + \text{nice} + \text{system} + \text{idle} + \text{iowait} + \text{irq} + \text{softirq} \quad (1)$$

$$\Delta \text{Total} = \text{TotalJiffies}_{t_2} - \text{TotalJiffies}_{t_1} \quad (2)$$

$$\Delta \text{Idle} = \text{idle}_{t_2} - \text{idle}_{t_1} \quad (3)$$

$$\text{CPU Utilization \%} = \left(1.0 - \frac{\Delta \text{Idle}}{\Delta \text{Total}} \right) \times 100 \quad (4)$$

3.2 Memory Mapping and Network Streaming Mechanisms

- **Memory Allocation Metrics:** Memory performance maps are evaluated by opening and parsing `/proc/meminfo`. The parser streams key data nodes, isolating `MemTotal`, `MemFree`, `Buffers`, and `Cached` to derive active resource utilization layouts via the engineering formula:

$$\text{Memory Used} = \text{MemTotal} - (\text{MemFree} + \text{Buffers} + \text{Cached})$$

4 Modular Software Engineering Layout

The development paradigm relies strictly on standard modular compilation patterns, splitting presentation code from core logic kernels to maintain high code maintainability:

Source Module	Architectural Domain	Operational Responsibilities
<code>stats.h</code>	Unified Architectural Blueprint	Global structures, definitions, and protocol layouts.
<code>stats.c</code>	Hard Logic Processing Layer	Active <code>/proc</code> scraping, file streaming, and parsing loops.
<code>main.c</code>	Presentation and UI Engine	Terminal cell allocation, window mapping, and signal t

Figure 1: Structural Separation of Project Modules

5 Low-Level Graphics Deployment via Ncurses

The application leverages the low-level processing capabilities of the `ncurses` library to transform standard command-line interfaces into structured, color-mapped visualization dashboards. By allocating logical sub-windows and updating window cell blocks sequentially, the program eliminates typical terminal flickering and optimizes screen redraw actions.

Specific color profiles are assigned to give clear visual weight to different system blocks: red accents emphasize volatile processing trends (CPU loads), while green boundaries track physical memory configurations (RAM).

6 Compilation Management and Strict Diagnostic Controls

6.1 Case Study: Silencing Unused Parameter Warnings under Strict Rules

To guarantee maximum optimization and catch structural flaws early, compilation flags inside the project `Makefile` enforce aggressive diagnostic rules (`-Wall -Wextra`).

A development challenge occurred when binding the system interruption signal (`SIGINT`) to clear terminal states before terminating the program. The signature required by the standard `signal.h` handler contract mandates an integer parameter representing the caught signal. However, because our tear-down logic does not actively process the value of the integer variable, the compiler raised strict unused parameter diagnostics.

To eliminate this without changing the standard signature or dropping our strict tracking flags, an explicit cast was applied to the parameter within the function frame:

Listing 1: Graceful Termination Script with Explicit Diagnostic Silencing

```
1 void handle_exit(int sig) {  
2     (void) sig; // Explicit cast to silence unused parameter  
3     // diagnostic warnings  
4     endwin(); // Unallocate ncurses window objects and reset  
5     // terminal layers  
6     exit(0); // Return operational control back to shell  
7     // environment  
8 }
```

This approach satisfies the requirements of the POSIX signal runtime, guarantees zero build-time warnings, and prevents runtime bugs during cleanup.

6.2 Production Makefile Script

The structural build and linker instructions are handled automatically by an optimized production Makefile:

Listing 2: Automated Production Makefile Layout

```
1 CC=gcc  
2 CFLAGS=-Wall -Wextra  
3 LIBS=-lncurses  
4  
5 TARGET=mahdi_monitor  
6  
7 all: $(TARGET)  
8  
9 $(TARGET): main.c stats.c  
10     $(CC) $(CFLAGS) main.c stats.c -o $(TARGET) $(LIBS)  
11  
12 clean:  
13     rm -f $(TARGET)
```

7 Conclusion and Next Phase Systems Engineering

The completed software infrastructure delivers stable, low-latency performance loops, rendering crisp hardware states directly within Arch Linux environments. By taking full control of native system resources, low-level directory streams, and advanced signaling interfaces, the tool shows how high performance can be achieved with a minimal resource footprint.

With user-space system monitoring fully implemented, our next engineering sprint will advance directly into bare-metal software development: designing and building an enterprise-grade Secure Bootloader ecosystem compliant with industry-standard cryptographic architectures.